Offline Goal-conditioned Model-based Reinforcement Learning in Pixel-based Environment

Seongsu Kim
Dept. Electrical Engineering
Hanyang University
Seoul, Republic of Korea
kimseongsu@hanyang.ac.kr

Jun Moon
Dept. Electrical Engineering
Hanyang University
Seoul, Republic of Korea
junmoon@hanyang.ac.kr
Corresponding author

***Abstract*—Model-based Reinforcement Learning (RL), with its capacity to learn and utilize a dynamics model for planning, stands out for its enhanced data efficiency, particularly in robotics, compared to its model-free RL. Despite these advancements RL algorithms commonly face challenges related to data inefficiency and the complexity of formulating reward functions. To address this issues, offline RL emerges as a promising approach, training policies from preexisting data without online interactions. However, a preexisting data will inevitably fail to encompass the full state-action space, potentially causing significant extrapolation errors. Furthermore, offline RL requires fully labeled data, which can be costly to acquire in large quantities. Goal-conditioned RL utilizes reward-free data, which can alleviate limitations of offline RL. In this paper, we demonstrate how to combine offline goal-conditioned RL with model-based RL to solve complex tasks in robotics. Our evaluation using image observations indicates that our method could effectively tackle real world tasks.**

***Index Terms*—Robotics, Offline RL, Goal-conditioned Learning, Model-based Learning, Hierarchical Learning**

I. Introduction

Deep Reinforcement Learning has successfully performed complex tasks in various environments, such as game, locomotion, and dexterous manipulation. The model-based RL [1], which learns a dynamics model and leverages this model for planning, offers significant benefits in robotics due to its higher data efficiency compared to model-free RL. Model-based approaches can predict future states and plan actions accordingly, which is crucial for applications requiring precise control and foresight, such as robotic manipulation and autonomous driving. However, RL algorithms still suffer from notorious data inefficiency and challenges with reward function formulation. Data inefficiency means that RL algorithms require a vast amount of interactions with the environment to learn effective policies, which can be impractical in real-world scenarios where data collection is expensive and time-consuming. Additionally, designing suitable reward functions that accurately reflect desired behaviors is often complex and task-specific, leading to sub-optimal learning outcomes.

Recently, offline RL, which trains policies from preexisting data without online interaction, mitigates the problem of data inefficiency [2]. Offline RL enables the reuse of previously collected datasets, significantly reducing the need for new data. However, offline RL requires entirely labeled data, which can be a limiting factor as it necessitates the availability of high-quality datasets with comprehensive reward annotations.

To tackle this limitation, goal-conditioned RL is proposed as a framework for training offline data with reward-free data [3]. Goal-conditioned RL shifts the focus from predefined rewards to achieving specific goals, thus providing flexibility in how tasks are learned and executed. This approach allows for more generalized policies that can adapt to various tasks without extensive reward engineering. Previous research in goal-conditioned RL has presented a range of techniques, such as hindsight relabeling [4], which involves reinterpreting past experiences to better understand goal achievement; contrastive learning [5], which distinguishes between successful and unsuccessful attempts; and hierarchical learning [6], which structures policies into multiple levels for handling complex tasks.

In this paper, we leverage hierarchical learning to solve long horizon tasks by decomposing them into simpler tasks. To summarized, we demonstrate how to combine offline goal-conditioned RL with model-based RL to solve complex tasks in robotics. We evaluate our method with image observations, which show that it has the potential to solve real-world tasks.

II. Preliminaries

A. Problem Setting

We consider the problem of offline goal-conditioned RL, formulized by Markov Decision Processes (MDPs) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mu, p, r)$ and a dataset $\mathcal{D}$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, $\mu \in \mathcal{P}(S)$ denotes an initial state distribution, $p \in \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathcal{S})$ denotes a transition dynamics distribution, and $r(s, g)$ denotes a goal-conditioned reward function. In this paper, we operate under the assumption that the goal space $\mathcal{G}$ coincides with the state space $\mathcal{S}$. The dataset $\mathcal{D}$ is comprised of trajectories τ, represented as sequences: $\{s_0, a_0 \cdots, s_T, a_T\}$. We aim to train an optimal goal-conditioned policy $\pi(a|s, g)$ from dataset $\mathcal{D}$.

B. Implicit Q-learning (IQL)

A primary difficulty in offline RL is a tendency of a policy to take an advantage of inflated value estimates for actions that fall outside the distribution. To handle this, [2] introduced implicit Q-learning (IQL), which circumvents the need to evaluate actions beyond the sample set by transforming the maximum operator in the Bellman equation into an expectile regression. IQL trains a state-action value function $Q_{\theta_Q}(s,a)$ and a state-value function $V_{\theta_V}(s)$ with the following equations:

$$\mathcal{L}_V(\theta_v) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[L_2^\tau(Q_{\hat{\theta}_Q}(s,a) - V_{\theta_V}(s))\right] \tag{1}$$

$$\mathcal{L}_V(\theta_Q) = \mathbb{E}_{(s,a,s')\sim\mathcal{D}}\left[\left(r(s,a) + \gamma V_{\theta_V} - Q_{\theta_Q}(s,a)\right)^2\right] \tag{2}$$

where $\hat{\theta}_Q$ denotes the target Q network parameters, and L_2^τ is the expectile loss with a parameter $\tau \in [0.5, 1) : |\tau - \mathbf{1}(x < 0)|x^2$. Expectile loss imposes a higher penalty on positive values compared to negative values. As the parameter τ approaches 1, the value function V_{θ_V} increasingly approximates the maximum of $Q_{\hat{\theta}_Q}(s,a)$. In IQL extracts the policy with advantage weighted regression (AWR) [7]:

$$\begin{aligned}&\mathcal{L}_\pi(\theta_\pi) = \\ &\mathbb{E}_{(s,a)\sim\mathcal{D}}[\exp\left(\beta(Q_{\hat{\theta}_Q}(s,a) - V_{\theta_V}(s))\right)\log \pi_{\theta_\pi}(a|s)]\end{aligned} \tag{3}$$

where $\beta \in [0, \infty)$ is an inverse temperature. Intuitively, (3) motivates the policy to choose actions that result in high Q values while staying close to the behavior of the data collection policy.

In this paper, we use a reward-free dataset and adopt an action-free variant of IQL [8] to train a value function conditioned on goals V_{θ_V}:

$$\begin{aligned}&\mathcal{L}_V(\theta_V) = \\ &\mathbb{E}_{(s,s')\sim\mathcal{D}, g\sim\mathcal{G}}[L_2^\tau(r(s,g) + \gamma V_{\hat{\theta}_V}(s',g) - V_{\theta_V}(s,g))]\end{aligned} \tag{4}$$

where $\hat{\theta}_V$ denotes the parameters of the target V network, and L_2^τ is the expectile loss.

III. Method

A. Hierarchical Implicit Q-learning

In complex and long-horizon environment, directly addressing goal-reaching tasks frequently presents challenges. [6] divide the learning of policies into two levels: a high-level policy $\pi^h_{\theta_h}(s_{t+k}|s_t, g)$ that generates intermediate sub-goals and a low-level policy $\pi^l_{\theta_l}(a_t|s_t, s_{t+k})$ that executes primitive actions to achieve these sub-goals. HIQL extracts two policies from the same value function. To train action-free value function, we sample the goals from either future states, random states, or the current state with probabilities of 0.3, 0.5, and 0.2 respectively, following [6]. After training the action-free value function with (3), HIQL extracts the policies with AWR:

$$\begin{aligned}&J_{\pi^h}(\theta_h) = \\ &\mathbb{E}_{(s_t, s_{t+k}, g)}[\exp(\beta \cdot \hat{A}^h(s_t, s_{t+k}, g)) \log \pi^h_{\theta_h}(s_{t+k}|s_t, g)]\end{aligned} \tag{5}$$

$$\begin{aligned}&J_{\pi^l}(\theta_l) = \\ &\mathbb{E}_{(s_t, a_t, s_{t+k})}[\exp(\beta \cdot \hat{A}^l(s_t, a_t, s_{t+k})) \log \pi^l_{\theta_l}(a_t|s_t, s_{t+k})]\end{aligned} \tag{6}$$

where β denotes the inverse temperature hyperparameter, and we approximate $\hat{A}^h(s_t, s_{t+k}, g)$ as $V_{\theta_V}(s_{t+k}, g) - V_{\theta_V}(s_t, g)$ and $\hat{A}^l(s_t, a_t, s_{t+k})$ as $V_{\theta_V}(s_{t+1}, s_t) - V_{\theta_V}(s_t, s_{t+k})$. Fig.1 shows the system architecture of the high-level policy and the low-level policy.

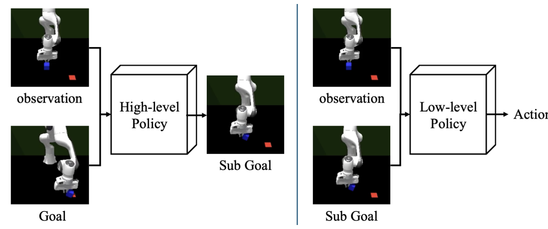

Fig. 1. System architecture of hierarchical policies

B. Model-based Reinforcement Learning

We train a dynamics model to predict the next state. The dynamics model is trained by the following loss:

$$\mathcal{L}_d(\theta_d) = \mathbb{E}_{(s,a,s')\sim\mathcal{D}}[\|d_\theta(s,a) - s'\|_2^2] \tag{7}$$

Note that (7) indicates that with the state and action, we can approximate the next state. We predict the next state based on the action and select the action that would maximize the value of the predicted state. In this paper, we predict three future steps and aim to maximize the sum of the value function.

C. Representations for high-dimensional observations

We evaluate in a pixel-based environment where the observation is an image and robot joint states. However, in high-dimensional domains, directly predicting an next state and sub-goals can be infeasible. In this paper, we encode image observations into a latent states, denoted as z, which are 512-dimensional. The high-level policy generates z_{t+k}, while the dynamics model generates z_{t+1}.

IV. Experiments

A. Baselines

Our experiments test our algorithm against three baselines: (1) TD-MPC algorithm is a model-based RL framework that leverages temporal difference learning to enhance model predictive control (MPC). (2) GC-IQL, which is a goal-conditioned variant of IQL, and (3) our algorithm without using dynamics model.

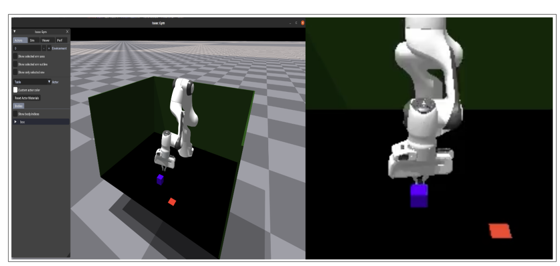

Fig. 2. Push cube environment and an image observation

B. Environment: Push Cube Task

We constructed the push cube environment in IsaacGym [9]. When the robot pushes the blue cube onto the red plate using a closed gripper, the task is complete. The action space of this task is 3, moving only along the x, y, and z axes, and the training was conducted using a dataset of 700 expert trajectories without any reward.

C. Results

Table 1 show the success rate of four algorithms, indicating that our algorithm achieves the best performance in our environment. Specifically, compared to our algorithm without a dynamics model, we observed a significant difference, confirming that utilizing a dynamics model for planning has a substantial effect.

D. Hyperparameters

In Table 2, we present the hyperparameters used in our experiment. We use the ResNet [10] architecture as an encoder to embed image observations into latent vectors. The latent vectors are 512-dimensional. We use layer normalization [11] only for MLP layers in the value network.

TABLE I
SUCCESS RATE COMPARISON

	TD-MPC	GC-IQL	Ours (w/o MBRL)	Ours
Success rate	18.19%	36.3%	66.27%	79.2%

TABLE II
HYPERPARAMETERS

Hyperparameter	Value
Value MLP dimensions	(512, 512, 512)
Policy MLP dimensions	(512, 512, 512)
Activation function	RELU [12]
Optimizer	Adam [13]
Representation architecture	ResNet CNN
Learning rate	0.0003
# gradient steps	100000
Batch size	256

V. CONCLUSION

We proposed an algorithm that combines offline goal-conditioned RL and model-based RL. We demonstrated that our algorithm can solve complex and long-horizon tasks without a reward function and with image observations. One drawback of our algorithm is that the planning process is time-consuming. This is a critical issue when implementing it in a real environment. Therefore, reducing the planning time will be our future work.

ACKNOWLEDGMENT

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2024-00397400), in part by the National Research Foundation (NRF) funded by the Korean government (MSIT) (No. RS-2024-00421129), in part by the Korea Institute of Energy Technology Evaluation and Planning(KETEP) and the Ministry of Trade, Industry & Energy(MOTIE) of the Republic of Korea (RS-2024-00422103), and in part by Institute of Information & communications Technology Planning and Evaluation (IITP) grant funded by the Korea government (Ministry of Science and ICT, MSIT) (RS-2020-II201373).

REFERENCES

[1] N.Hansen, X.Wang, and H.Su. Temporal difference learning for model predictive control. In International Conference on Machine Learning (ICML),2022.

[2] I. Kostrikov, A. Nair, and S. Levine. Offline reinforcement learning with implicit q-learning. In International Conference on Learning Representations (ICLR),2022.

[3] T. Schaul, D. Horgan, K. Gregor, and D. Silver. Universal value function approximators. In International Conference on Machine Learning (ICML),2015.

[4] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba. Hindsight experience replay. In Neural Information Processing Systems (NeurIPS),2017.

[5] B. Eysenbach, T. Zhang, R. Salakhutdinov, and S. Levine. Contrastive learning as goal-conditioned reinforcement learning. In Neural Information Processing Systems (NeurIPS),2022.

[6] P. Seohong, D. Ghosh, B. Eysenbach, and S. Levine. Hiql: Offline Goal-conditioned rl with latent states as actions. In Neural Information Processing Systems (NeurIPS),2024.

[7] X. Peng, A. Kumar, G. Zhang, and S. Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. ArXiv,abs/1910.00177,2019.

[8] H. Xu, L. Jiang, J. Li, and X. Zhan. A policy-guided imitation approach for offline reinforcement learning. In Neural Information Processing Systems (NeurIPS),2022.

[9] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State. Isaac Gym: High performance gpu-based physics simulation for robot learning. In Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2),2021.

[10] K. He, X, Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp.770-778).

[11] J. Ba, J. Kiros, and G. Hinton. Layer normalization. ArXiv,abs/1607.06450,2016

[12] A. Agarap. Deep learning using rectified linear units (relu). ArXiv,abs/180.308375,2018

[13] D. Kingma, and J. Ba. Adam: A method for stochastic optimization. In International Conference on Learning Representations (ICLR),2015.